

Chapter 5 (Process API) — Homework (Simulation)

Zhiyuan Pan

All runs use `fork.py` in `ostep-homework/cpu-api/`. Trees are the simulator's own `-c` output.

Q1

Run `./fork.py -s 10` and see which actions are taken. Can you predict what the process tree looks like at each step? Use the `-c` flag to check your answers. Try some different random seeds (`-s`) or add more actions (`-a`) to get the hang of it.

```
./fork.py -s 10 -c:
```

Action: a forks b

```
a
└─ b
```

Action: a forks c

```
a
├─ b
└─ c
```

Action: c EXITS

```
a
└─ b
```

Action: a forks d

```
a
├─ b
└─ d
```

Action: a forks e

```
a
├─ b
├─ d
└─ e
```

Two rules predict every step: `X forks Y` adds `Y` as a new child of `X`, and `Y EXITS` removes `Y` from the tree. My predictions matched `-c` for this seed and for other seeds. The only case that needs extra care is a process that exits while it still has children (see Q4).

Q2

One control the simulator gives you is the `fork_percentage`, controlled by the `-f` flag. The higher it is, the more likely the next action is a fork; the lower it is, the more likely the action is an exit. Run the simulator with a large number of actions (e.g., `-a 100`) and vary the `fork_percentage` from 0.1 to 0.9. What do you think the resulting final process trees will look like as the percentage changes? Check your answer with `-c`.

Measured with `./fork.py -s 1 -a 100 -f F -F -c`:

-f	processes alive at end	max depth	direct children of a
0.1	3	1	2
0.3	5	2	3
0.5	17	3	8
0.7	45	3	25
0.9	91	7	8

- **Low fork percentage (0.1–0.3):** most actions are exits, so processes die almost as soon as they are created. The final tree is tiny and flat. With `-f 0.1` it is just:

```

a
├─ z
└─ y

```

- **Middle (0.5–0.7):** the tree grows, but many processes end up directly under `a`. One reason is that every orphan is re-parented to `a` (see Q4), which keeps the tree wide and shallow.
- **High (0.9):** almost every action is a fork, and the parent is picked at random from *all* live processes, so children keep forking their own children. The final tree is large (91 processes) and much deeper (depth 7).

Q3

Now, switch the output by using the `-t` flag (e.g., run `./fork.py -t`). Given a set of process trees, can you tell which actions were taken?

Example `./fork.py -s 5 -t` shows these trees, one after each action:

```

a → a — b → a → a — c → a → a — d

```

The actions are therefore:

a forks b, b EXITS, a forks c, c EXITS, a forks d

(confirmed with `-c`). Yes, when every intermediate tree is shown, the actions can always be recovered exactly. Only one action happens between two consecutive trees: a new node `Y` under `X` means `X forks Y`, and a node `Y` that disappears means `Y EXITS`.

Q4

One interesting thing to note is what happens when a child exits; what happens to its children in the process tree? To study this, let's create a specific example: `./fork.py -A a+b,b+c,c+d,c+e,c-`. This example has process 'a' create 'b', which in turn creates 'c', which then creates 'd' and 'e'. However, then, 'c' exits. What do you think the process tree should like after the exit? What if you use the `-R` flag? Learn more about what happens to orphaned processes on your own to add more context.

Just before `c` exits:

```
a
├── b
│   ├── c
│   │   ├── d
│   │   └── e
```

After `c EXITS` (default):

```
a
├── b
├── d
└── e
```

`d` and `e` become **orphans**, and they are re-parented to the root `a`, not to their grandparent `b`. This matches real Unix: when a process exits, the kernel gives its children to `init` (PID 1, `systemd` on Ubuntu). `init`

periodically calls `wait()` on its adopted children, so when an orphan exits, its zombie entry is cleaned up instead of staying in the process table forever.

With `-R ( ./fork.py -A a+b,b+c,c+d,c+e,c- -R -c )`:

```
a
└─ b
   └─ d
      └─ e
```

Now the orphans are adopted by the exiting process's parent `b`. Linux can do something similar through a *subreaper*: a process that calls `prctl(PR_SET_CHILD_SUBREAPER, 1)` adopts orphaned descendants instead of `init`. `systemd --user` and container runtimes use this.

Q5

One last flag to explore is the `-F` flag, which skips intermediate steps and only asks to fill in the final process tree. Run `./fork.py -F` and see if you can write down the final tree by looking at the series of actions generated. Use different random seeds to try this a few times.

Seed 3 (`./fork.py -s 3 -a 8 -F`):

```
a forks b, b forks c, a forks d, d forks e,
b forks f, c EXITS, d EXITS, a forks g
```

`c` exits with no children. `d` exits while it still has child `e`, so `e` is re-parented to `a`. Final tree (confirmed with `-c`):

```
a
├─ b
│  └─ f
├─ e
└─ g
```

Seed 4 (`./fork.py -s 4 -a 8 -F`):

a forks b, a forks c, b forks d, d EXITS,
a forks e, a forks f, f forks g, f EXITS

f exits while it has child g, so g moves to a. Final tree:

```
a
├─ b
├─ c
├─ e
└─ g
```

Seed 2 (`./fork.py -s 2 -a 8 -F`):

a forks b, b EXITS, a forks c, c forks d,
a forks e, c forks f, f EXITS, d EXITS

Final tree:

```
a
├─ c
└─ e
```

The key to getting these right is the re-parenting rule from Q4. Without it, e in seed 3 or g in seed 4 would wrongly be drawn under the process that exited.

Q6

Finally, use both `-t` and `-F` together. This shows the final process tree, but then asks you to fill in the actions that took place. By looking at the tree, can you determine the exact actions that took place? In which cases can you tell? In which can't you tell? Try some different random seeds to delve into this question.

Seed 5 (`./fork.py -s 5 -t -F`) — final tree:

```
a
└─ d
```

Names are handed out in order (`b` , `c` , `d` , ...), so the missing letters tell us `b` and `c` existed and exited. But the tree alone cannot tell who forked them or when they exited. The real answer is

`a+b` , `b-` , `a+c` , `c-` , `a+d` , yet `a+b` , `a+c` , `b-` , `c-` , `a+d` or `a+b` , `b+c` , `c-` , `b-` , `a+d` would produce exactly the same final tree.

Seed 1 (`./fork.py -s 1 -t -F`) — final tree:

```
a
├─ b
├─ e
└─ d
```

This looks as if `a` forked `b` , `d` and `e` . The real sequence is `a+b` , `a+c` , `c+d` , `a+e` , `c-` : `c` forked `d` , and `d` was re-parented to `a` when `c` exited. We can tell that `c` existed (its letter is missing), but not whether `d` was forked by `a` or by `c` . The child order is a clue: `d` is listed after `e` , which hints that it was attached to `a` late, i.e. that it was re-parented. Still, the exact sequence cannot be determined.

- **We can tell exactly** when no process ever exited. Then every node's parent in the tree is the process that forked it, and the alphabetical order of the names gives the order of the forks.
- **We cannot tell** once exits are involved:
 - An exited process leaves no node in the tree, only a gap in the names.
 - Re-parenting hides who really forked an orphan.
 - The exact time of each exit relative to the other actions is lost.